Day 18 - Part-2 – Terraform & IaC Fundamentals for AWS

1. The core idea: Infrastructure as Code

Infrastructure as Code (IaC) means defining cloud infrastructure in version-controlled files rather than manually creating resources through the AWS Console.

For a GenAI SaaS platform, infrastructure may include:

```
Route 53 DNS
      ↓
Application Load Balancer
      ↓
EKS cluster
      ↓
FastAPI RAG services
      ├── RDS PostgreSQL: metadata, users, conversations
      ├── Vector database / pgvector: embeddings
      ├── ElastiCache Redis: cache, sessions, rate limits
      ├── S3: source documents and processed artifacts
      ├── ECR: container images
      └── Secrets Manager: API keys and credentials
```

Terraform allows this complete stack to be reviewed, tested, reproduced, and deployed from code. Terraform compares the configuration, stored state, and real infrastructure to determine the required changes. (HashiCorp Developer)

---

2. Imperative vs declarative infrastructure

Imperative approach

An imperative approach describes **how to create infrastructure step by step**:

```
aws ec2 create-vpc ...
aws eks create-cluster ...
aws s3api create-bucket ...
aws rds create-db-instance ...
```

The engineer controls the order and must handle:

- Existing resources
- Partial failures
- Update logic
- Dependencies
- Rollback
- Duplicate execution

Declarative approach

Terraform describes the **desired final state**:

```
resource "aws_s3_bucket" "documents" {
  bucket = "rag-prod-documents"
}
```

You are saying:

> "This bucket should exist with this configuration."

Terraform constructs a dependency graph, compares the desired configuration with the current infrastructure, and calculates the operations needed to reconcile them.

Interview explanation

> Terraform is declarative. I describe the desired infrastructure, and Terraform determines the API operations and dependency order needed to reach that state.

---

3. Terraform vs AWS Console vs CloudFormation

Approach	Strength	Limitation
AWS Console	Fast for learning and debugging	Hard to reproduce, review, audit, or scale
AWS CLI/ scripts	Flexible and automatable	You must implement idempotency and update logic
CloudFormation	Deep AWS-native integration and managed stacks	Primarily AWS-focused; YAML can become verbose
Terraform	Consistent workflow across AWS and other systems; strong module ecosystem	Requires careful state and provider management

CloudFormation also provides declarative IaC through YAML or JSON templates and manages collections of AWS resources as stacks. Terraform becomes attractive when a platform must manage AWS alongside systems such as GitHub, Kubernetes, Datadog, Cloudflare, or other SaaS providers through one workflow. (AWS Documentation)

When I would choose Terraform

Choose Terraform when:

- The organization already standardizes on Terraform.
- Infrastructure spans AWS and non-AWS systems.
- Reusable modules are important.
- Teams want a consistent `plan → review → apply` workflow.
- Infrastructure is managed through pull requests.

Choose CloudFormation or CDK when:

- The platform is entirely AWS-native.
- The company prefers AWS-managed stacks.
- Teams need immediate support for very new AWS capabilities.
- Existing governance is based on CloudFormation StackSets and change sets.

The senior-level answer is not "Terraform is always better." It is:

> Choose the tool that fits the organization’s cloud footprint, skills, governance model, and existing delivery platform.

---

4. Where Terraform fits in a RAG/LLM platform

A practical GenAI deployment commonly has several layers.

Foundation layer

Terraform creates:

- AWS accounts and IAM roles
- VPC
- Public and private subnets
- Internet Gateway and NAT Gateway
- Route tables
- Security groups
- VPC endpoints
- KMS keys

Platform layer

Terraform creates:

- EKS cluster
- EKS managed node groups
- Optional GPU node groups
- ECR repositories
- AWS Load Balancer Controller IAM roles
- CloudWatch log groups
- Secrets Manager secret containers

Data layer

Terraform creates:

- S3 buckets for documents
- RDS PostgreSQL
- ElastiCache Redis
- OpenSearch or managed vector database connectivity
- Backup and retention policies

Application delivery layer

A good division of responsibility is:

```
Terraform:
  AWS infrastructure and durable platform resources

Helm / Kubernetes manifests / GitOps:
  FastAPI Deployment, Service, Ingress, HPA and application configuration
```

```
CI/CD:
  Build image → push to ECR → update deployment → run tests
```

Terraform can manage Kubernetes and Helm resources, but using Terraform for every application image release often couples infrastructure changes with high-frequency application deployments. My usual recommendation is to use Terraform for the cluster and supporting AWS resources, and Helm or GitOps for application releases.

---

5. Terraform language fundamentals

Terraform uses **HashiCorp Configuration Language**, or HCL.

Provider

A provider translates Terraform resources into calls to an external API.

```
terraform {
  required_version = ">= 1.10.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

The version constraints above are illustrative rather than a requirement. In a real repository, test provider upgrades deliberately and commit `.terraform.lock.hcl`, which records the selected provider versions. The AWS provider is maintained through the Terraform Registry, and HashiCorp recommends defining provider and Terraform version constraints for collaborative configurations. (Terraform Registry)

Version constraint intuition

```
version = "~> 6.0"
```

Means:

```
Allow compatible 6.x versions
Do not automatically move to 7.x
```

For highly controlled production platforms, some teams pin an exact version:

```
version = "6.55.0"
```

This improves reproducibility but requires deliberate upgrades.

Resource block

A `resource` block creates or manages infrastructure.

```
resource "aws_ecr_repository" "rag_api" {
  name                 = "${var.environment}-rag-api"
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }
}
```

Reference:

```
aws_ecr_repository.rag_api.repository_url
```

General format:

```
resource_type.resource_name.attribute
```

Data block

A `data` block reads existing infrastructure but does not create it.

```
data "aws_route53_zone" "main" {
  name         = "example.com"
  private_zone = false
}
```

Examples:

- Find an existing Route 53 hosted zone
- Read the current AWS account ID
- Find an AMI
- Read an existing VPC
- Retrieve Secrets Manager metadata

A Terraform data source queries provider-managed information without provisioning the corresponding infrastructure object. (HashiCorp Developer)

Variable block

Variables are the inputs to a module.

```
variable "environment" {
  description = "Deployment environment"
  type        = string

  validation {
    condition     = contains(["dev", "stage", "prod"], var.environment)
    error_message = "Environment must be dev, stage, or prod."
```

```
  }
}

variable "aws_region" {
  type    = string
  default = "ap-south-1"
}
```

Environment-specific values can be provided in a `.tfvars` file:

```
# environments/prod/terraform.tfvars

environment = "prod"
aws_region  = "ap-south-1"
vpc_cidr    = "10.30.0.0/16"
```

Local values

`locals` calculate or centralize internal values.

```
locals {
  name_prefix = "ai60-${var.environment}"

  common_tags = {
    Project     = "rag-platform"
    Environment = var.environment
    ManagedBy   = "terraform"
    Owner       = "ai-platform"
  }
}
```

Use them elsewhere:

```
resource "aws_s3_bucket" "documents" {
  bucket = "${local.name_prefix}-documents"
  tags   = local.common_tags
}
```

Variables are external inputs. Locals are internal calculated values. Terraform locals can reference variables, resources, functions, and other local values. (HashiCorp Developer)

Output block

Outputs expose useful resource attributes.

```
output "document_bucket_name" {
  value = aws_s3_bucket.documents.id
}

output "ecr_repository_url" {
  value = aws_ecr_repository.rag_api.repository_url
}
```

Outputs are similar to function return values. Child modules use them to expose information to parent modules, while root modules can expose values to the CLI or automation systems. (HashiCorp Developer)

---

6. Terraform workflow

`terraform init`

```
terraform init
```

It:

- Initializes the working directory
- Configures the backend
- Downloads providers
- Downloads modules
- Creates or updates `.terraform.lock.hcl`

Terraform requires initialization before normal planning and application operations. (HashiCorp Developer)

`terraform fmt`

```
terraform fmt -recursive
```

Formats HCL consistently.

`terraform validate`

```
terraform validate
```

Checks structural correctness.

`terraform plan`

```
terraform plan -var-file=terraform.tfvars
```

Shows the proposed changes:

```
+ create
~ update in place
-/+ destroy and recreate
- destroy
```

A production pipeline commonly saves the reviewed plan:

```
terraform plan \
  -var-file=terraform.tfvars \
  -out=tfplan
```

`terraform apply`

```
terraform apply tfplan
```

Executes exactly the saved plan.

Applying a saved plan is safer in CI/CD than running an unreviewed `terraform apply -auto-approve`. Terraform documents this two-step plan-and-apply pattern for automation. (HashiCorp Developer)

`terraform destroy`

```
terraform destroy -var-file=terraform.tfvars
```

Deletes resources tracked by the current state.

Use this carefully. It is useful for disposable development environments but should be tightly restricted in production. Terraform destroy creates and executes a destroy plan for resources managed in the current workspace. (HashiCorp Developer)

---

7. Terraform state

What is `terraform.tfstate`?

Terraform state maps configuration addresses to actual cloud resources.

For example:

```
aws_s3_bucket.documents
        ↓
arn:aws:s3:::ai60-prod-documents
```

Without state, Terraform would not reliably know:

- Which real resource corresponds to each HCL block
- Resource IDs and attributes
- Dependency metadata
- Which resources it manages
- Whether a change needs an update or replacement

Terraform refreshes its understanding of managed infrastructure before operations and uses state to determine the necessary changes. (HashiCorp Developer)

Why local state is dangerous

By default, Terraform may create:

```
terraform.tfstate
terraform.tfstate.backup
```

Local state creates several problems:

- Only one developer has the latest copy.
- Two engineers can apply concurrently.

- The file may be accidentally committed.
- Sensitive values may appear in plain text.
- A damaged or deleted laptop may lose the state.

Do not commit state files to Git. Terraform recommends using storage that supports secure access and state locking instead of normal version control. (HashiCorp Developer)

---

8. Remote backend: S3 state

A production backend can store Terraform state in S3.

```
terraform {
  backend "s3" {
    bucket       = "ai60-terraform-state"
    key          = "rag-platform/prod/network/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

Recommended S3 bucket protections include:

- Block public access
- Server-side encryption
- Bucket versioning
- Restricted IAM access
- Audit logging
- Separate bootstrap ownership
- Deletion protection where appropriate

Modern locking vs DynamoDB locking

Historically, the standard answer was:

```
backend "s3" {
  bucket         = "ai60-terraform-state"
  key            = "prod/terraform.tfstate"
  region         = "ap-south-1"
  dynamodb_table = "terraform-locks"
  encrypt        = true
}
```

That pattern used a DynamoDB table to prevent simultaneous writes.

The modern backend option is:

```
use_lockfile = true
```

HashiCorp's current documentation marks DynamoDB-based locking as deprecated and recommends the S3 lockfile capability. Both may temporarily be configured during migration from older Terraform versions. (HashiCorp Developer)

Interview answer

> State locking prevents two pipelines or engineers from writing the same state simultaneously. In current Terraform, I would prefer S3 native lockfiles where supported. I would recognize S3 plus DynamoDB locking as the established legacy architecture.

Bootstrapping problem

Terraform needs an S3 bucket to store state, but the S3 bucket itself may also be created by Terraform.

Solve this with a separate bootstrap configuration:

```
infra/
├── bootstrap/
│   └── creates-state-bucket-and-KMS
└── live/
    └── uses-that-remote-backend
```

The bootstrap state is small and highly controlled. It should rarely change.

Never manually edit state

Do not open `terraform.tfstate` and change JSON by hand.

Use commands such as:

```
terraform state list
terraform state show aws_s3_bucket.documents
terraform state mv ...
terraform state rm ...
terraform import ...
```

Manual editing can corrupt relationships between configuration and infrastructure.

---

9. Drift

Drift means the real AWS infrastructure no longer matches Terraform configuration and state.

Example:

1. Terraform sets the EKS node group minimum size to 2.
2. Someone changes it to 5 in the AWS Console.
3. The next plan detects the difference.

```
Console value:    5
Terraform config: 2
```

Terraform may plan to restore it to 2.

Drift-management practices

- Restrict manual console changes.

- Run scheduled read-only plans.
- Review CloudTrail events.
- Import legitimate manually created resources.
- Update HCL when an emergency manual fix must become permanent.
- Do not blindly apply a drift-correction plan.

A senior engineer first determines **why** drift occurred before reconciling it.

---

10. Modules and reusable structure

A module is a collection of Terraform resources managed together. Every Terraform directory is a module. The directory where Terraform commands run is the **root module**; modules called from it are **child modules**. (HashiCorp Developer)

Suggested repository structure

```
infra/
├── modules/
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── eks/
│   ├── rds/
│   ├── elasticache/
│   ├── s3-documents/
│   ├── ecr/
│   └── dns/
│
├── live/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   └── terraform.tfvars
│   ├── stage/
│   └── prod/
│
└── bootstrap/
    └── state-backend/
```

Root module composition

```
module "network" {
  source = "../../modules/vpc"

  name       = local.name_prefix
  vpc_cidr   = var.vpc_cidr
  az_count   = 3
  nat_mode   = var.environment == "prod" ? "per_az" : "single"
  common_tags = local.common_tags
}
```

```
module "eks" {
  source = "../../modules/eks"

  cluster_name       = "${local.name_prefix}-eks"
  vpc_id             = module.network.vpc_id
  private_subnet_ids = module.network.private_subnet_ids
  environment        = var.environment
}

module "database" {
  source = "../../modules/rds"

  name               = "${local.name_prefix}-postgres"
  vpc_id             = module.network.vpc_id
  private_subnet_ids = module.network.database_subnet_ids
  application_sg_id  = module.eks.application_security_group_id
}
```

This root module expresses architecture. Child modules hide low-level details such as IAM roles, subnet groups, security groups, logging, encryption, and alarms.

Module design principle

A good module represents a reusable capability:

```
Good:
  secure-postgres
  private-eks-cluster
  encrypted-document-bucket

Too small:
  one-security-group-rule
  one-subnet

Too large:
  entire-company-cloud
```

Avoid creating an abstraction for every resource. Create modules when multiple resources form a meaningful unit with a clear lifecycle.

---

11. Dev, stage, and production environments

There are two common approaches.

Option A: Terraform workspaces

```
terraform workspace new dev
terraform workspace new stage
terraform workspace new prod
```

Each workspace has separate state.

You might use:

```
locals {
  environment = terraform.workspace
}
```

Advantages

- Less duplicated configuration
- Convenient for temporary or similar environments
- Simple developer workflow

Risks

- All environments share the same configuration and backend definition.
- It is easier to accidentally select the wrong workspace.
- Large environment differences become full of conditions.
- Access control per environment can become difficult.

Terraform workspaces separate state for one configuration, but they are not the same thing as fully isolated repositories, accounts, or deployment boundaries. (HashiCorp Developer)

Option B: separate root directories and states

```
live/dev
live/stage
live/prod
```

Each environment has:

- Separate backend key
- Separate variables
- Separate plan
- Separate approval process
- Preferably a separate AWS account

Example keys:

```
rag-platform/dev/terraform.tfstate
rag-platform/stage/terraform.tfstate
rag-platform/prod/terraform.tfstate
```

Recommendation

For long-lived production platforms:

> Use reusable child modules with separate environment root modules, separate state, and ideally separate AWS accounts.

Use workspaces for:

- Developer sandboxes
- Short-lived preview environments
- Environments with almost identical topology
- Cases where organizational controls already make workspace selection safe

---

12. Naming and tagging conventions

Naming

```
locals {
  name_prefix = "${var.project}-${var.environment}-${var.aws_region}"
}
```

Example resources:

```
rag-prod-ap-south-1-eks
rag-prod-ap-south-1-documents
rag-prod-ap-south-1-postgres
```

Tags

```
locals {
  common_tags = {
    Project     = "rag-platform"
    Environment = var.environment
    Owner       = "ai-platform"
    ManagedBy   = "terraform"
    CostCenter  = "genai"
    DataClass   = "internal"
  }
}
```

Useful tags provide:

- Cost allocation
- Ownership
- Environment identification
- Security classification
- Automation targeting
- Incident response context

---

13. GenAI example: VPC and EKS

A production EKS network typically places:

```
Public subnets:
  Internet-facing ALB
  NAT Gateways

Private application subnets:
  EKS worker nodes
  FastAPI pods

Private database subnets:
  RDS
  ElastiCache
```

A simplified root configuration might look like:

```
module "vpc" {
  source = "../../modules/vpc"

  name = local.name_prefix
  cidr = "10.30.0.0/16"

  availability_zones = [
    "ap-south-1a",
    "ap-south-1b",
    "ap-south-1c"
  ]

  public_subnet_cidrs = [
    "10.30.0.0/24",
    "10.30.1.0/24",
    "10.30.2.0/24"
  ]

  private_subnet_cidrs = [
    "10.30.10.0/24",
    "10.30.11.0/24",
    "10.30.12.0/24"
  ]
}

module "eks" {
  source = "../../modules/eks"

  cluster_name       = "${local.name_prefix}-eks"
  vpc_id             = module.vpc.vpc_id
  private_subnet_ids = module.vpc.private_subnet_ids

  general_node_group = {
    instance_types = ["m7i.large"]
    min_size       = 2
    desired_size   = 3
    max_size       = 10
  }
}
```

The EKS child module would normally create:

- Cluster IAM role
- EKS cluster
- Managed node groups
- Node IAM roles
- Security groups
- Control-plane logging
- OIDC provider for workload IAM
- Cluster add-ons

- KMS encryption configuration

Amazon EKS is AWS's managed Kubernetes service. For HTTP applications, the AWS Load Balancer Controller can create an Application Load Balancer from Kubernetes Ingress resources and provide Layer 7 routing. (Amazon Web Services, Inc.)

GenAI-specific node groups

A RAG service might use:

```
General CPU nodes:
  FastAPI APIs
  retrieval services
  rerank API clients
  ingestion workers

Memory-optimized nodes:
  document parsing
  large in-memory indexing jobs

GPU nodes:
  self-hosted embedding models
  rerankers
  vLLM/TGI inference
```

Apply Kubernetes taints and tolerations so normal API pods do not consume expensive GPU nodes.

---

14. S3 bucket for RAG documents

```
resource "aws_s3_bucket" "documents" {
  bucket = "${local.name_prefix}-documents"
  tags   = local.common_tags
}

resource "aws_s3_bucket_versioning" "documents" {
  bucket = aws_s3_bucket.documents.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "documents" {
  bucket = aws_s3_bucket.documents.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "documents" {
```

```
  bucket = aws_s3_bucket.documents.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Possible object prefixes:

```
raw/
processed/
chunks/
evaluation-datasets/
failed-ingestion/
```

A lifecycle policy can move old source documents or failed ingestion artifacts to cheaper storage, subject to business retention requirements.

---

15. RDS PostgreSQL for metadata

RDS may store:

- Users and tenants
- Document metadata
- Conversation metadata
- Prompt configuration
- Evaluation results
- Ingestion job state
- Chunk-to-document relationships
- Optionally embeddings through `pgvector`

Simplified example:

```
resource "aws_db_subnet_group" "rag" {
  name       = "${local.name_prefix}-db-subnets"
  subnet_ids = module.vpc.database_subnet_ids
}

resource "aws_db_instance" "rag" {
  identifier = "${local.name_prefix}-postgres"

  engine         = "postgres"
  engine_version = "17"
  instance_class = var.db_instance_class

  allocated_storage     = 100
  max_allocated_storage = 500
  storage_encrypted     = true

  db_name  = "rag"
  username = "rag_admin"
```

```
  manage_master_user_password = true

  db_subnet_group_name   = aws_db_subnet_group.rag.name
  vpc_security_group_ids = [aws_security_group.database.id]

  multi_az               = var.environment == "prod"
  backup_retention_period = var.environment == "prod" ? 14 : 3

  deletion_protection = var.environment == "prod"
  skip_final_snapshot  = var.environment != "prod"

  tags = local.common_tags
}
```

Important design point:

> Terraform provisions PostgreSQL, networking, backups, and security. Database migration tooling should create schemas, tables, indexes, and the `pgvector` extension.

Do not make Terraform responsible for continuous application schema migrations.

---

16. ElastiCache Redis

Redis may support:

- Semantic result caching
- Exact prompt-response caching
- Session storage
- Rate limiting
- Distributed locks
- Background-job coordination
- Short-lived retrieval results

A conceptual module call:

```
module "redis" {
  source = "../../modules/elasticache"

  name               = "${local.name_prefix}-redis"
  subnet_ids         = module.vpc.database_subnet_ids
  application_sg_id  = module.eks.application_security_group_id

  transit_encryption = true
  at_rest_encryption = true
  multi_az           = var.environment == "prod"
}
```

Do not expose RDS or Redis publicly. Permit access only from approved application security groups or private network paths.

---

17. ECR for application images

```
resource "aws_ecr_repository" "rag_api" {
  name                 = "${local.name_prefix}/rag-api"
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "KMS"
  }

  tags = local.common_tags
}
```

Release pipeline:

```
1. Run tests
2. Build FastAPI Docker image
3. Scan image
4. Push immutable image tag to ECR
5. Update Kubernetes deployment
6. Wait for readiness
7. Run smoke tests
8. Promote or rollback
```

Prefer immutable identifiers:

```
rag-api:git-a81f92c
```

Avoid relying only on:

```
rag-api:latest
```

AWS publishes patterns combining Terraform-created ECR repositories with automated image-building pipelines. (AWS Documentation)

---

18. DNS and Route 53

Request flow

```
api.example.com
      ↓
Route 53 hosted zone
      ↓
Alias record
      ↓
Application Load Balancer
      ↓
EKS Ingress
```

```
      ↓
FastAPI Service
      ↓
FastAPI Pods
```

Route 53 alias records can point directly to an Elastic Load Balancer, including for the root domain where a normal CNAME cannot be used. (AWS Documentation)

Route 53 alias record

Assume the ALB information is exported by an ingress or load-balancer module:

```
data "aws_route53_zone" "main" {
  name         = "example.com"
  private_zone = false
}

resource "aws_route53_record" "api" {
  zone_id = data.aws_route53_zone.main.zone_id
  name    = "api.example.com"
  type    = "A"

  alias {
    name                   = var.alb_dns_name
    zone_id                = var.alb_zone_id
    evaluate_target_health = true
  }
}
```

---

19. TLS and ACM certificate

Request an ACM certificate

```
resource "aws_acm_certificate" "api" {
  domain_name       = "api.example.com"
  validation_method = "DNS"

  lifecycle {
    create_before_destroy = true
  }

  tags = local.common_tags
}
```

Create DNS validation records

```
resource "aws_route53_record" "certificate_validation" {
  for_each = {
    for option in aws_acm_certificate.api.domain_validation_options :
    option.domain_name => {
      name   = option.resource_record_name
      record = option.resource_record_value
```

```
      type   = option.resource_record_type
    }
  }

  zone_id = data.aws_route53_zone.main.zone_id
  name    = each.value.name
  type    = each.value.type
  ttl     = 60
  records = [each.value.record]
}
```

Complete certificate validation

```
resource "aws_acm_certificate_validation" "api" {
  certificate_arn = aws_acm_certificate.api.arn

  validation_record_fqdns = [
    for record in aws_route53_record.certificate_validation :
    record.fqdn
  ]
}
```

ACM DNS validation creates CNAME records that prove domain ownership and allow managed certificate renewal while the records remain available. An HTTPS ALB listener must have an appropriate TLS certificate. (AWS Documentation)

The certificate ARN is then associated with the ALB listener, commonly through AWS Load Balancer Controller annotations on the Kubernetes Ingress.

---

20. Production release and traffic patterns

Rolling deployment

Kubernetes gradually replaces old pods with new pods.

```
v1 v1 v1
   ↓
v1 v1 v2
   ↓
v1 v2 v2
   ↓
v2 v2 v2
```

Use:

- Readiness probes
- Liveness probes
- Pod disruption budgets
- `maxUnavailable`
- `maxSurge`
- Automated rollback

Canary deployment

Send a small portion of traffic to the new version:

```
95% → stable version
 5% → canary version
```

Observe:

- Error rate
- P95/P99 latency
- Token generation latency
- Retrieval success
- LLM provider errors
- Cost per request
- Hallucination or quality regression
- User feedback

For fine-grained canaries, application-layer routing, ingress controllers, ALB weighted target groups, or a service mesh are generally more responsive than DNS.

DNS-weighted release

Route 53 weighted records can divide DNS responses:

```
api.example.com → blue ALB, weight 90
api.example.com → green ALB, weight 10
```

Conceptual Terraform:

```
resource "aws_route53_record" "blue" {
  zone_id        = data.aws_route53_zone.main.zone_id
  name           = "api.example.com"
  type           = "A"
  set_identifier = "blue"

  weighted_routing_policy {
    weight = 90
  }

  alias {
    name                   = var.blue_alb_dns_name
    zone_id                = var.blue_alb_zone_id
    evaluate_target_health = true
  }
}
```

Create a corresponding green record with weight `10`.

Route 53 supports weighted, failover, and latency-based routing policies. Weighted routing is useful for coarse blue/green transitions, while failover routing supports active-passive architecture and latency routing supports multi-Region traffic placement. (AWS Documentation)

DNS limitation

DNS changes are not perfectly immediate because clients and resolvers cache responses according to TTL and sometimes beyond it.

Therefore:

- Use DNS weighting for regional or environment-level traffic shifts.
- Use ALB, ingress, or service-mesh routing for rapid request-level canaries.
- Keep the previous deployment available during rollback.
- Do not delete the old environment immediately after changing DNS.

---

21. Secrets handling

Never hardcode secrets

Bad:

```
variable "openai_api_key" {
  default = "sk-real-secret-value"
}
```

Also bad:

```
resource "aws_db_instance" "rag" {
  password = "SuperSecret123"
}
```

Preferred design

Terraform creates:

- Secrets Manager secret metadata
- IAM permissions
- KMS key associations
- Kubernetes workload identity

The application retrieves the secret at runtime.

```
resource "aws_secretsmanager_secret" "llm_api_key" {
  name = "/rag/${var.environment}/llm/api-key"
  tags = local.common_tags
}
```

The secret value should be populated through a protected delivery mechanism rather than committed in Terraform code.

AWS recommends storing credentials and other sensitive information in Secrets Manager, using encryption, limiting access, rotating secrets, and monitoring secret usage. (AWS Documentation)

Important Terraform detail

```
variable "password" {
  type      = string
```

```
  sensitive = true
}
```

`sensitive = true` mainly hides the value from normal CLI output. It does **not automatically guarantee that the value is absent from state**.

Terraform state can contain sensitive data in plain text. Newer Terraform capabilities include ephemeral values and provider-supported write-only arguments, which can prevent selected values from being persisted in plans and state. (HashiCorp Developer)

Senior-level answer

> I treat Terraform state as sensitive data even when variables are marked sensitive. I encrypt remote state, tightly restrict access, avoid passing secret values through Terraform where possible, and let workloads retrieve secrets at runtime through IAM roles.

For EKS, use workload IAM rather than static AWS access keys inside pods.

---

22. Real-world example 1: complete FastAPI RAG environment

Infrastructure

Terraform provisions:

```
VPC
├── Public subnets
│   └── ALB
├── Private application subnets
│   └── EKS nodes and FastAPI pods
└── Private database subnets
    ├── RDS PostgreSQL
    └── ElastiCache Redis

Additional:
├── S3 document bucket
├── ECR repository
├── Route 53 DNS
├── ACM certificate
├── Secrets Manager
└── CloudWatch logging
```

Runtime workflow

```
1. User sends query to api.example.com.
2. Route 53 resolves to ALB.
3. ALB forwards HTTPS traffic to EKS Ingress.
4. FastAPI validates authentication.
5. Service checks Redis cache.
6. Retriever queries the vector store.
7. Metadata is loaded from PostgreSQL.
8. Relevant chunks are passed to an LLM.
9. Response is streamed to the client.
10. Metrics and trace identifiers are recorded.
```

Terraform makes this environment reproducible. Application code remains responsible for retrieval, generation, evaluation, and request-level behavior.

---

23. Real-world example 2: dev, stage, and production

Use the same child modules but different root inputs.

Development

```
environment       = "dev"
eks_min_nodes     = 1
eks_max_nodes     = 3
db_instance_class = "db.t4g.small"
multi_az_database = false
single_nat_gateway = true
```

Stage

```
environment       = "stage"
eks_min_nodes     = 2
eks_max_nodes     = 5
db_instance_class = "db.m7g.large"
multi_az_database = false
```

Production

```
environment        = "prod"
eks_min_nodes      = 3
eks_max_nodes      = 20
db_instance_class  = "db.r7g.large"
multi_az_database  = true
deletion_protection = true
```

The architecture stays consistent while capacity, availability, retention, and protection settings differ.

A mature promotion workflow is:

```
PR opened
   ↓
fmt + validate + lint + security scan
   ↓
terraform plan for dev
   ↓
apply dev
   ↓
integration tests
   ↓
plan stage
   ↓
approval + apply stage
   ↓
load/evaluation tests
   ↓
```

```
plan prod
   ↓
manual approval
   ↓
apply prod
```

Do not copy the production state into stage. Promote code and reviewed configuration, not state files.

---

24. Real-world example 3: preview environments

For each pull request:

```
PR #482
   ↓
Create isolated namespace or temporary stack
   ↓
Deploy FastAPI branch image
   ↓
Run RAG regression tests
   ↓
Destroy after PR closes
```

Possible names:

```
pr-482-rag-api
pr-482-documents
pr-482.example.internal
```

For cost control, preview environments may share durable infrastructure such as an EKS cluster while receiving isolated:

- Kubernetes namespace
- Database schema
- Redis prefix
- S3 prefix
- DNS record

Creating an entirely separate VPC, EKS cluster, and RDS instance for every pull request is usually too slow and expensive.

---

25. Terraform best practices

State

- Use a remote backend.
- Enable locking.
- Encrypt state.
- Enable S3 versioning.
- Restrict state access.
- Separate state by system and lifecycle.
- Never commit state to Git.

Providers

- Set Terraform and provider version constraints.
- Commit `.terraform.lock.hcl`.
- Upgrade providers through reviewed pull requests.
- Test upgrade plans outside production.

Modules

- Keep modules focused.
- Pin external module versions.
- Expose only useful outputs.
- Validate important variables.
- Avoid deeply nested module chains.
- Include examples and documentation.

Environments

- Prefer separate AWS accounts for strong isolation.
- Use separate state for long-lived environments.
- Keep names and tags consistent.
- Do not make prod behavior depend entirely on workspace selection.

Security

- Use IAM roles rather than static credentials.
- Keep databases and caches private.
- Use KMS encryption.
- Store secrets in Secrets Manager.
- Treat plan files and state files as sensitive.
- Run policy and security checks in CI.

Delivery

- Run `fmt`, `validate`, linting, security scanning, and `plan`.
- Review replacement operations carefully.
- Save and apply the reviewed plan.
- Require approval for production.
- Record which commit produced each apply.

---

26. Common pitfalls

1. One massive state file

A single state containing VPC, EKS, databases, DNS, monitoring, and every application becomes:

- Slow
- High-risk
- Hard to grant access to
- Difficult for multiple teams to modify

Split state by ownership and lifecycle, not arbitrarily by individual resource.

Example:

```
network
platform
data
observability
application-foundation
```

2. Overusing `-target`

```
terraform apply -target=aws_eks_cluster.main
```

`-target` is useful for exceptional recovery situations but can skip related changes and leave an incomplete desired state. It should not become the normal deployment workflow.

3. Ignoring replacement indicators

This line is critical:

```
-/+ resource must be replaced
```

Replacement of an S3 bucket, RDS instance, EKS cluster, or security-sensitive resource may cause downtime or data loss.

4. Hardcoded resource names

Hardcoded global names cause collisions:

```
bucket = "documents"
```

Use predictable environment-aware names.

5. Secrets in `.tfvars`

A `.tfvars` file containing production passwords is still a plaintext secret file. Adding it to `.gitignore` is not a complete secrets-management strategy.

6. Unpinned external modules

An uncontrolled module upgrade can change dozens of resources. Pin and upgrade explicitly.

7. Using Terraform as a shell-script runner

Avoid excessive:

```
provisioner "local-exec" { ... }
```

Terraform is strongest when providers manage resource lifecycles. Provisioners are difficult to model, retry, and reverse safely.

8. Mixing infrastructure and application releases

Changing an image tag should not require reviewing unrelated VPC or RDS changes. Separate infrastructure cadence from application release cadence.

9. Terraforming database schemas

Terraform should provision RDS. Use Alembic, Flyway, Liquibase, or another migration tool for application schemas.

10. Manual emergency change never returned to code

After an emergency console fix:

1. Stabilize the service.
2. Update Terraform configuration.
3. Review the plan.
4. Reconcile state and infrastructure.

Otherwise, the next apply may undo the fix.

---

27. Senior AI Engineer design perspective

For an AI platform, Terraform decisions affect more than infrastructure consistency.

They affect:

- **Latency:** Region selection, network path, NAT usage, cache placement.
- **Availability:** Multi-AZ databases, node groups, DNS failover.
- **Cost:** GPU nodes, NAT gateways, idle clusters, storage lifecycle.
- **Security:** private subnets, workload IAM, secret access.
- **Model delivery:** ECR, GPU scheduling, inference endpoints.
- **Data governance:** S3 encryption, retention, tenant isolation.
- **Reproducibility:** identical evaluation and release environments.
- **Rollback:** retaining the old application and infrastructure version.

A strong interview answer connects Terraform to these product-level consequences rather than discussing only HCL syntax.

---

28. Interview Q&A

Q1. What problem does Terraform state solve?

Answer:

Terraform state maps resource addresses in the configuration to real infrastructure objects. It also stores resource attributes and dependency metadata. Terraform uses that information to calculate the difference between the desired configuration and the deployed infrastructure.

For teams, I store state remotely with encryption, access control, versioning, and locking.

Q2. Why is state locking important?

Answer:

Without locking, two engineers or CI pipelines could read the same initial state and simultaneously write conflicting updates, potentially corrupting state or infrastructure.

Historically, S3 backends commonly used DynamoDB for locking. Current Terraform supports native S3 lockfiles through `use_lockfile`, and DynamoDB locking is deprecated. (HashiCorp Developer)

Q3. Would you use Terraform workspaces for dev, stage, and prod?

Answer:

It depends on the isolation requirements.

Workspaces are acceptable when environments are almost identical and the organization has controls preventing accidental workspace selection. For long-lived production systems, I prefer separate root modules, separate state, separate CI approval paths, and preferably separate AWS accounts.

That provides clearer access control and reduces blast radius.

Q4. How would you structure Terraform for a RAG platform?

Answer:

I would create reusable child modules for:

```
VPC
EKS
RDS
Redis
S3
ECR
DNS and ACM
IAM and observability
```

Then I would create separate environment root modules that compose those capabilities.

I would also separate states according to lifecycle and ownership—for example, networking, platform, data, and application foundation—so changing an ECR repository does not lock or endanger the RDS and VPC state.

Q5. How do you handle secrets in Terraform?

Answer:

I avoid passing secret values through Terraform whenever possible.

Terraform creates Secrets Manager resources, IAM policies, and workload identities. Applications retrieve secret values at runtime. I treat state as sensitive because marking a variable `sensitive` hides it from normal output but may not remove it from state.

Where supported, I use ephemeral values, write-only provider arguments, or service-managed credentials such as RDS-managed master passwords.

Q6. How do you detect and handle infrastructure drift?

Answer:

I run regular Terraform plans and monitor out-of-band AWS changes. When drift appears, I first determine whether it was accidental, an emergency fix, or a legitimate change.

Then I either:

- Reapply the Terraform configuration,
- Update HCL to represent the accepted change,
- Import the resource,
- Or reconcile state with Terraform state commands.

I do not blindly apply because Terraform might reverse a valid production fix.

Q7. Should Terraform deploy the FastAPI application to EKS?

Answer:

Terraform can deploy Kubernetes and Helm resources, but I normally separate responsibilities.

Terraform provisions durable infrastructure such as EKS, IAM, ECR, RDS, S3, DNS, and the load-balancer foundation. Helm or GitOps manages high-frequency application releases.

This reduces coupling and lets application teams roll forward or back without running a broad infrastructure plan.

Q8. How would you release a new RAG API version safely?

Answer:

I would:

1. Build and scan an immutable container image.
2. Push it to ECR using a commit-based tag.
3. Deploy it as a canary.
4. Verify readiness and smoke tests.
5. Observe latency, error rate, retrieval quality, LLM failures, and cost.
6. Increase traffic gradually.
7. Roll back to the previous image if thresholds fail.

For request-level canaries, I prefer ingress or load-balancer weighting. I use Route 53 weighting for coarse environment or regional traffic shifts.

Q9. How would you connect Route 53, ACM, ALB, and EKS?

Answer:

Terraform requests an ACM certificate and creates Route 53 DNS validation records. The certificate is attached to an HTTPS listener on the ALB.

The AWS Load Balancer Controller creates or manages the ALB based on Kubernetes Ingress configuration. Route 53 then creates an alias record such as `api.example.com` pointing to the ALB.

The request path becomes:

```
Route 53 → HTTPS ALB → Kubernetes Ingress → Service → FastAPI pods
```

Q10. What would you inspect carefully in a production Terraform plan?

Answer:

I focus on:

- Resource replacements
- RDS, S3, EKS, IAM, and networking changes
- Security-group widening
- Public exposure
- Database deletion or snapshot behavior
- IAM privilege increases
- DNS and certificate changes
- Provider or module upgrades
- Changes caused by drift
- Unexpected resource counts

The key senior-engineer mindset is:

> A successful Terraform apply is not the objective. A safe, explainable, reversible infrastructure change is the objective.

Final recall summary

```
Terraform = declarative infrastructure management

Core HCL:
  resource = create/manage
  data     = read existing
  variable = input
  locals   = internal calculated values
  output   = exported values

Workflow:
  init → fmt → validate → plan → review → apply

State:
  maps Terraform configuration to real infrastructure
  use encrypted remote state and locking
  current S3 backend supports native lockfiles
  DynamoDB locking is now legacy/deprecated

Structure:
  reusable child modules
  thin environment root modules
  separate states by lifecycle and ownership

GenAI AWS stack:
  Route53 → ALB → EKS → FastAPI
  ECR for images
  S3 for documents
  RDS for metadata
```

```
  Redis for caching
  Secrets Manager for credentials

Production:
  immutable releases
  canary or blue/green
  HTTPS through ACM
  controlled DNS changes
  reviewed and reproducible infrastructure
```